National University of Computer and Emerging Sciences
FAST School of Computing — Karachi Campus
Spring 2026

Features Document

Courtify — Sports Court Booking & Management Platform

Course: CS3009 — Software Engineering
Instructor: Asst. Prof. Engr. Abdul Rahman
Version: 2.0 — Final
Date: May 2026

Prepared By
Muhammad Zayan Ahmed
Muhammad Hammad Uddin
[Zayan Ahmed]

Confidential — For academic evaluation only

Contents

Section	Title	Page
1	Document Purpose	2
2	Feature Mapping Matrix	2
3	Feature Group Coverage	3
3.1	Court & Owner Management	3
3.2	Booking & Payment	3
3.3	AI Chatbot & Matchmaking	3
3.4	Admin & Platform Support	4
4	Notes	4

1. Document Purpose

This document presents the implemented product features of the Courtify platform and maps each feature to its corresponding backend implementation, including API routes, service modules, and data models. It reflects the current working system as developed in the Spring 2026 semester and supports both academic evaluation and live demonstration.

Courtify is a full-stack sports court booking and management platform built on Flask (Python), React 18, and PostgreSQL, with Stripe payment integration, a RAG-based AI chatbot (LangChain + Pinecone + Google Gemini), and APScheduler-driven booking lifecycle automation.

2. Feature Mapping Matrix

ID	Feature	API / Module	Implementation Detail
F-01	User registration	POST /api/users/register	Creates user with bcrypt-hashed password; validates email uniqueness; role assigned (user/court_owner).
F-02	User login (JWT)	POST /api/users/login	Returns signed JWT access token (24h expiry) after credential verification with bcrypt comparison.
F-03	Auth profile	GET /api/users/current	Retrieves authenticated user profile; requires valid Bearer token via @jwt_required().
F-04	Update profile	PUT /api/users/update	Updates username, gender, or profile image; validates token ownership.
F-05	Register court	POST /api/courts/register	Court owner registers court; status defaults to pending until admin approves.
F-06	Browse approved courts	GET /api/courts/all/approved	Public endpoint; returns only admin-approved courts with full metadata.
F-07	Court detail	GET /api/courts/{id}	Returns single court or 404; includes hourly rate, schedule, and status.
F-08	Update/Delete court	PUT/DELETE /api/courts/{id}	Owner-only CRUD; role + ownership validated before modification.
F-09	Create booking	POST /api/bookings/create	Checks slot availability (DB UNIQUE constraint); creates Booking(pending); initiates Stripe session.
F-10	Approve/Reject booking	PUT /api/bookings/approve|reject/{id}	Court owner approves or rejects pending bookings; status updated in DB.
F-11	Cancel booking	PUT /api/bookings/cancel/{id}	Player cancels own booking; ownership validated; completed bookings cannot be cancelled.
F-12	View my bookings	GET /api/bookings/mybookings	Returns all bookings for the authenticated user; filtered by user_id.

ID	Feature	API / Module	Implementation Detail
F-13	Stripe checkout session	POST /api/payments/create-session	Creates Stripe Checkout Session with court/slot details; returns hosted payment URL.
F-14	Stripe webhook	POST /api/payments/webhook	Receives Stripe events; verifies HMAC signature; updates Payment + Booking on success.
F-15	AI chatbot query	POST /api/chat/ask	Passes user message + chat history to LangChain RAG chain; returns grounded answer.
F-16	Chatbot warmup	GET /api/chat/warmup	Pre-loads RAG pipeline (Pinecone + Gemini) into memory; reduces first-query latency.
F-17	Post match request	POST /api/matchmaking/create	Player posts sport, location, date/time, contact for matchmaking.
F-18	Browse match requests	GET /api/matchmaking/all	Returns all active match requests; requires authentication.
F-19	Delete match request	DELETE /api/matchmaking/{id}	Owner of match request can delete their own entry.
F-20	Submit review	POST /api/reviews/create	Authenticated user submits 1–5 star rating for a booked court.
F-21	View court reviews	GET /api/reviews/court/{id}	Returns all reviews for a specific court with ratings.
F-22	Add to favourites	POST /api/favourites/add	Adds court to user's favourites list; prevents duplicates.
F-23	View favourites	GET /api/favourites	Returns all courts favourited by the authenticated user.
F-24	Remove favourite	DELETE /api/favourites/{id}	Removes a court from the user's favourites.
F-25	Admin dashboard stats	GET /api/admin/dashboard-stats	Returns platform-wide metrics: users, courts, bookings, revenue.
F-26	Admin approve/reject court	PUT /api/admin/update-court-status/{id}	Admin sets court status to approved or rejected; approved courts become publicly visible.
F-27	Admin delete user	DELETE /api/admin/delete-user/{id}	Admin cascade-deletes user with all associated courts, bookings, payments, reviews.
F-28	Owner revenue report	GET /api/admin/revenue-report	Court owner views revenue analytics for their own courts only.
F-29	APScheduler auto-complete	Scheduler / booking_scheduler.py	Interval job (15 min): transitions past confirmed/approved bookings to completed status.
F-30	RBAC enforcement	Auth decorators	@role_required(role) on all sensitive routes; 403 on role mismatch.

3. Feature Group Coverage

3.1 Court & Owner Management

- Full court lifecycle: registration (pending) → admin approval → public listing → booking.
- Court owners have isolated CRUD access — cannot modify another owner's courts (403 enforced).
- Admin-controlled quality gate: no court is publicly visible until explicitly approved (status=approved).
- Owner revenue reporting provides booking count and revenue breakdown per court.

3.2 Booking & Payment

- Slot-based booking with DB-level UNIQUE constraint prevents double-booking at the database layer.
- Full booking lifecycle: pending → confirmed/approved → cancelled/completed (APScheduler automated).
- Stripe Checkout integration: hosted payment page, HMAC-verified webhook, no raw card data handled.
- Players can cancel bookings; court owners can approve or reject; system auto-completes past bookings.

3.3 AI Chatbot & Matchmaking

- RAG chatbot (LangChain + Pinecone + Google Gemini + HuggingFace embeddings) answers platform queries grounded in Courtify knowledge base documents.
- Warmup endpoint pre-loads the pipeline at startup; subsequent queries served in ~2s average.
- Multi-turn conversation supported via chat_history parameter passed on each request.
- Matchmaking module: players post sport/location/time requests and browse others' requests to find playing partners.

3.4 Admin & Platform Support

- Admin dashboard aggregates platform-wide statistics (users, courts, bookings, revenue).
- Admin can delete any user with full cascade (courts, bookings, payments, reviews, favourites).
- Reviews and ratings (1–5 stars) provide social proof for court discovery.
- Favourites allow players to bookmark preferred courts for quick re-access.
- JWT-based stateless auth with bcrypt password storage; CORS restricted to trusted frontend origin.

4. Notes

- This document reflects the current implemented system as of May 2026.
- Feature descriptions align with Flask-RESTX backend route namespaces and SQLAlchemy ORM models.
- Intended for academic evaluation and live demonstration to Asst. Prof. Engr. Abdul Rahman.

- Email notifications (TBD-01), Stripe refunds (TBD-02), i18n (TBD-03), and mobile app (TBD-04) are deferred to future releases as documented in the SRS.